

Final Project Report

B.I.G: A BST-Indexed Graph Model for Version Control

Course Name: Data Structures

GitHub Repository: <https://github.com/Eender753030/B.I.G>

Date: 2025 / 12 / 7

Contents

I. Problems	3
II. Development Environment:	3
III. Data Structures	3
BST (Binary Search Tree).....	3
The Implicit Merkle DAG (Directed Acyclic Graph).....	4
Stack	4
IV. System Architecture:	5
1. Command Layer (<code>src/commands/</code>)	5
2. Core Layer (<code>src/core/</code>)	5
3. Data Structure Layer (<code>src/ds/</code>)	5
4. Infrastructure Layer (<code>src/utils/</code>)	5
V. Implementation Details	6
Concepts in B.I.G	6
Project Directory	6
Index (Staging Area).....	6
Blob.....	6
Commit.....	6
Branch	6
Leader (HEAD).....	6
Solving Problem 1	7
Solving Problem 2	7
Solving Problem 3	7
VI. Code.....	8
VII. Results	11
VIII. Discussion	18
IX. Conclusion.....	19

I. Problems

My goal is to implement a version control system like Git. There are three major challenges that need to be solved:

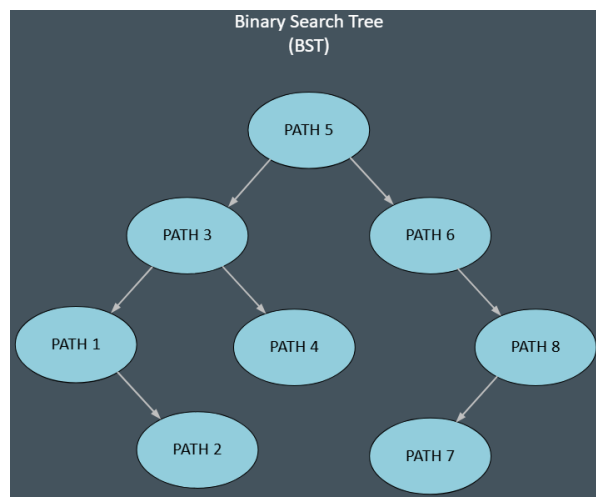
1. How to save every snapshot in each Commit without wasting disk space?
A naive copy-paste approach would grow non-linearly.
2. How to process large binary files like images or videos without data corruption caused by C string handling?
3. How to safely checkout to history commits without destroying the user's uncommitted work?

II. Development Environment:

I chose **Linux (WSL2)** as the platform for development and release, using **GCC (C17 Standard)** and **Make** for build management. Using **VS Code** for coding with **Git** and **GitHub** to version control this project.

III. Data Structures

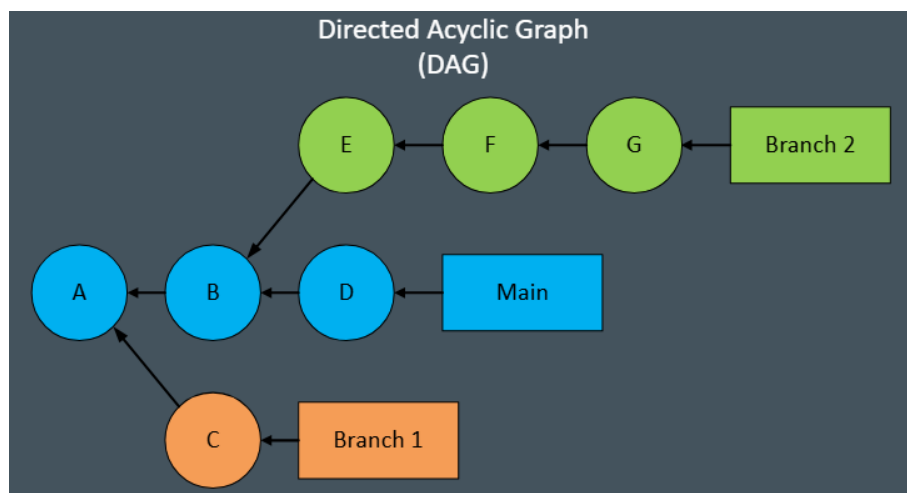
BST (Binary Search Tree)



- This is the main structure to store the file's path and corresponding hash ID.
- In a naive insertion order of files. It has a big chance to be a skewed tree. The worst case of search and insert is $O(n)$.

- But when the stage command ends. Using in-order traversal to save the list. Means the list of file path is a sorted list. Next time to load the list into BST. By using algorithm of sorted array to a **Balanced BST**. This algorithm uses divide-and-conquer. Every time we choose the middle one to become every tree or subtree's root. It is a $O(n)$ operation but can make sure BST is $O(\log n)$ search for every time. Do not need to implement AVT, Red-Black Tree.

The Implicit Merkle DAG (Directed Acyclic Graph)



- Unlike traditional graph problems, there is no real **Graph** structure in my code.
- It is an implicit graph in the system. Every commit point to its parent and link together. When branches out from the main line it will become a graph.
- The advantage is that we do not need to load the whole graph every time instead of printing out every log.

Stack

- A helper structure used in iteration of BST operations.
- To prevent system stack overflow for large tree by using recursive.

IV. System Architecture:

I divide the system into a Four-Layered Architecture.

1. Command Layer (`src/commands/`)

- The entry of the system. It handles user input (CLI arguments), parses flags (e.g., `-m`, `-d`). This layer executes the command task and calls the Core Layer.
- Example: `cmd_commit.c`, `cmd_status.c`

2. Core Layer (`src/core/`)

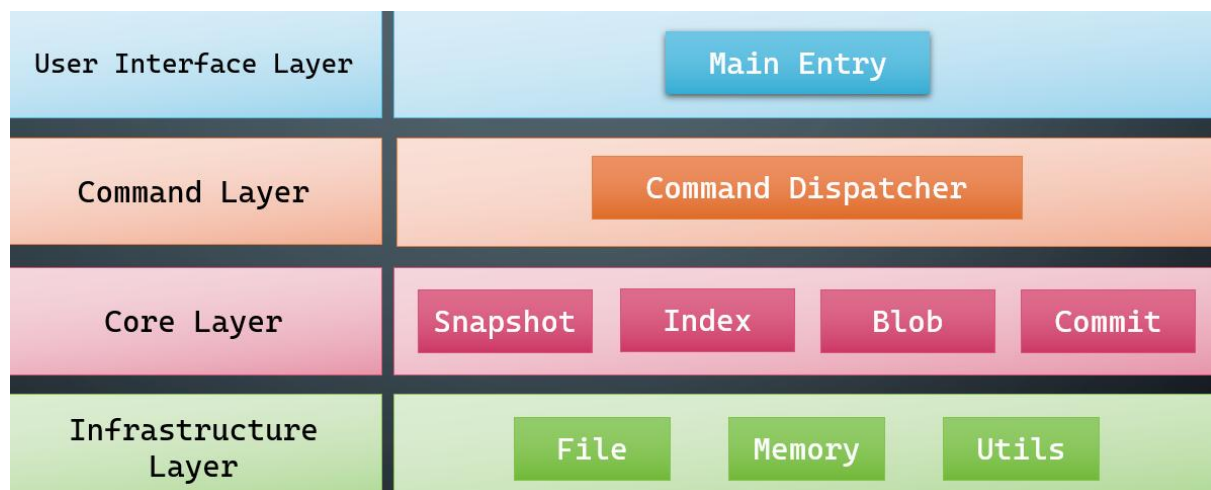
- Contains the main logic of version control system. It defines what a **Commit**, **Blob**, or **Index** is. Executes complex tasks like **snapshotting** or **committing**.
- Example: `commit.c` (manages DAG), `cmd_status.c` (calculates hash), `index.c` (manages staging area by using BST)

3. Data Structure Layer (`src/ds/`)

- Contains Generic data structures. These structures are completely independent of the version control system logic. Using `void*` pointers that make them reusable for any data type.
- Example: `bst.c` (Generic Binary Search Tree), `stack.c` (Generic Stack).

4. Infrastructure Layer (`src/utils/`)

- Handles low-level system operations like memory management, file I/O, and error handling.
- Example: `memory.c`, `file_handle.c`.



V. Implementation Details

Concepts in B.I.G

Here are some main concepts in B.I.G

Project Directory

The root directory containing the `.big/` folder. This is where the version control system stores all its metadata, objects, and references. The system automatically detects the project root no matter where the command is executed within the project.

Index (Staging Area)

Acts as a “Staging Area” between the project directory and the commit history. It maintains a list of files that are tracked. In `big/index` containing lines of file path and blob hash. When we run `big add`, we are updating this list.

Blob

The fundamental unit of data storage. B.I.G is a **Content-Addressable Storage (CAS)** system. The filename of a Blob is the SHA-1 (using DJB2 algorithm) hash of its content. This means if two files have the same content, they share the same Blob, effectively deduplicating storage.

Commit

A snapshot of the project at a specific point in time. It is treated as an object too but it is a folder that save snapshot list and metadata like log, datetime, and commit parent hash

Branch

A movable pointer to a specific commit. It allows independent lines of development. Stores in `.big/refs/` (e.g., `.big/refs/feature_api`). The file content is simply the Hash ID of the commit it points to. Creating a branch only creates a tiny file with a hash string, making it $O(1)$.

Leader (HEAD)

A special pointer that records “where we are”. Instead of pointing directly to a Commit Hash, it points to a reference (e.g., `refs/main`). This indirection allows us to implement Branching easily moving a Branch just means updating the reference, and Leader automatically follows.

These are solutions to previous problems

Solving Problem 1

- In this case. My first thought was to use copy mechanics to save all files which were staged in each Commit. When user staged files, B.I.G will create a list of staged files called “Index”. But the problem is, if user changes the files before committing. The commit result will be the new changed file, not the staged one. That is not a version control for.
- The second version is to save the staged files into Index as a directory. That can solve the changing file problem. However, in every Commit. B.I.G will copy a whole snapshot of current Index. The folder size will become bigger in each Commit nonlinearly same as first version.
- Those two methods waste a lot of space on disks to store those histories. In the final version. I use the concept of Blob (Binary Large Object). When user stages the files, B.I.G creates an object that is a snapshot of status of each file. And use the content of file to calculate a unique hash ID to represent this file’s snapshot. Index just need to record the file name and corresponding hash ID. So as Commit.

Solving Problem 2

- In binary files, there are not only characters like text files. There is value 0 in it. It means when using string handle like `strlen()`. It will terminate automatically when encountering value 0 (The `\0` is 0) which can cause binary file stop in early processing. Making the Blob a corrupted file.
- The solution is to use file I/O function in `<stdio.h>` header. And I implemented functions like `get_file_len()` by using `fseek()` and `ftell()` cooperation to get the real size of files.

Solving Problem 3

- My implementation for checkout. First, delete all files in current project directory. And then restore all files in the Commit Index. This is a dangerous operation. If users did not commit the files before checkout. They will lose those files forever.
- The solution is to prohibit this behavior. If there are uncommitted and unstaged files. The operation will be cancelled. This is a stupid but safest way to protect user data.

VI. Code

Here I show five represent codes in this project (Some code be omitted).

1. Generic Data Structure

```
/* Generic Binary Search Tree
 * For the BST data structure can be reused in future
 * I use void* pointer to store data,
 * and function pointer (cmp_callback) can let user decide how to compare data to build BST
 * That can make bst.c is completely independent, totally no need to know like "Index" or "Commit"
 */
bst_t* bst_create(int8_t (*cmp_callback)(void*, void*)) {
    bst_t* new_bst = xmalloc(sizeof(*new_bst));

    new_bst->root = NULL;
    new_bst->cmp_callback = cmp_callback;
    new_bst->amount = 0;
    return new_bst;
}
```

2. Create Blob and Binary Safety

```
// Create blob can make BST doesn't need to hold big content all the time that cause overhead
// Just need to hold the hash for future use
char *blob_create_from_file(const char *path) {
    // Use read binary mode that can read all types of file
    FILE *file = xfopen(path, "rb");

    uint64_t file_len = get_file_len(file); // get_file_len to malloc() enough space
    char *content = xmalloc(file_len);
    if (file_len > 0) {
        // Use fread with length make sure it will not terminate by 0 value in binary file
        if (fread(content, 1, file_len, file) != file_len) {
            fclose(file);
            error_custom_msg("Failed to read file: %s\n", path);
        }
    }
    fclose(file);

    /*
    ... (Hash calculation) ...
    */

    // Write whole content into blob
    if (access(object_path, F_OK) != 0) {
        FILE *obj_file = fopen(object_path, "wb"); // Use write binary
        if (obj_file == NULL) {
            error_custom_msg("Failed to write object file: %s\n", object_path);
        }
        uint64_t write_bytes = fwrite(content, 1, file_len, obj_file);
        if (write_bytes != file_len) {
            fclose(obj_file);
            error_custom_msg("Error: Write object content failed\n");
        }
        fclose(obj_file);
    }

    xfree(content);

    return hash_str;
}
```

3. The Implicit Graph

```
/* The Implicit Graph
 * Here is where the Implicit Graph start formation
 * Every node simply record its parent
 */
void save_commit_obj(commit_node_t *node) {
    /*
     * ... (Combine commit object path and write amount into list) ...
     */

    // The metadata
    FILE *info_file = xfopen(buffer, "w");
    // First line is datetime
    fprintf(info_file, "%s\n", node->datetime);
    // Second line is for parent, if no parent, write null instead
    // Here create the DAG's edge
    if (node->parent != NULL) {
        fprintf(info_file, "%s\n", node->parent->commit_hash);
    } else {
        fprintf(info_file, "%s\n", "null");
    }
    // Write whole log into it
    fprintf(info_file, "%s\n", node->log);
    fclose(info_file);

    xfree(index_content);
}
```

4. Recursive Directory Traversal

```
/* Recursive Directory Traversal
 * We need to scan directory in commands 'add', 'status', and 'checkout'
 * There are three different BST use in those commands
 * I use <dirent.h> with recursive to traversal directory
 * And use <sys/stat.h> to judge is file or directory
 */
static void _process_handler(snapshot_bst_t *bst, const char *root_path, bool on_file, bool on_dir,
                             bool have_leader, void *args) {
    // Open root directory
    DIR *dir = opendir(root_path);
    /*
     * ... (Error handle and declare structure) ...
     */

    // Start to checking all files and directories
    while ((file_dirent = readdir(dir)) != NULL) {
        /*
         * ... (Skip . and .. and .big and big)
         * ... (Combine the path)
         */

        // Get file's or directory's status
        if (stat(pathbuffer, &file_stat) == -1) {
            errno_handle(__func__, __FILE__, __LINE__);
        }
        // Check is directory
        if (S_ISDIR(file_stat.st_mode)) {
            // If is, keep go into the directory
            _process_handler(bst, pathbuffer, on_file, on_dir, have_leader, args);
            // Parse the action on directory
            if (on_dir == true) {
                _process_parser(bst, pathbuffer, NULL, on_dir, have_leader, args);
            }
        } else {
            // This means is a file, parse the action on file
            if (on_file == true) {
                _process_parser(bst, pathbuffer, on_file, NULL, have_leader, args);
            }
        }
    }
    closedir(dir); // Use closedir() instead of free()
}
```

5. The Add Workflow

```
void cmd_add(int argc, char *argv[]) {
    /*
     ... (Error check, initialization and args parse) ..
    */

    // Back to project directory and save working directory first
    char *org_dir = NULL;
    cd_to_project_root(&org_dir);

    // Get a builded BST from index file
    // This is a balanced BST (empty BST for first time)
    snapshot_bst_t *snapshot_bst = read_index_file();

    /*
     ... (Load Leader BST) ...
    */

    // Main process of user's input
    struct stat file_stat;
    for (uint64_t i = 0; i < input_size; i++) {
        // Get the relative path of file to project directory
        char *normalized_path = relative_path_calc(org_dir, root_path_list[i]);
        if (delete_mode == true) {
            // delete mode for 'big add -d'
            delete_file_in_dir(normalized_path, snapshot_bst, true);
        } else {
            // Normal add file
            process_dir_or_file(normalized_path, snapshot_bst, &file_stat, leader_bst);
        }
        xfree(normalized_path);
    }

    // Save BST data to index file
    save_index_file(snapshot_bst);

    xfree(org_dir);
    snapshot_bst_free(&leader_bst);
    snapshot_bst_free(&snapshot_bst);
}
```

VII. Results

Note: This project can only be executed on Linux.

First, we need to know how to build the executable file and use it. Using make in **B.I.G/** to one step compile and link all source and header files.

```
(base) eender@DESKTOP-Eender:~/Workspace/Study/Course/Data Structure/Project/B.I.G$ make
Compile succeeded -> build/utils/utils.o
gcc -O2 -Wall -Wextra -Wconversion -Wpedantic -Werror -g -fsanitize=address -Iinclude
Compile succeeded -> build/utils/error_handle.o
gcc -O2 -Wall -Wextra -Wconversion -Wpedantic -Werror -g -fsanitize=address -Iinclude
Compile succeeded -> build/utils/file_handle.o
gcc -O2 -Wall -Wextra -Wconversion -Wpedantic -Werror -g -fsanitize=address -Iinclude
Compile succeeded -> build/utils/memory.o
gcc -fsanitize=address build/main.o build/core/blob.o build/core/index.o build/core/
ommands/cmd_init.o build/commands/cmd_status.o build/commands/cmd_branch.o build/ds/stack
Link succeeded -> bin/big
```

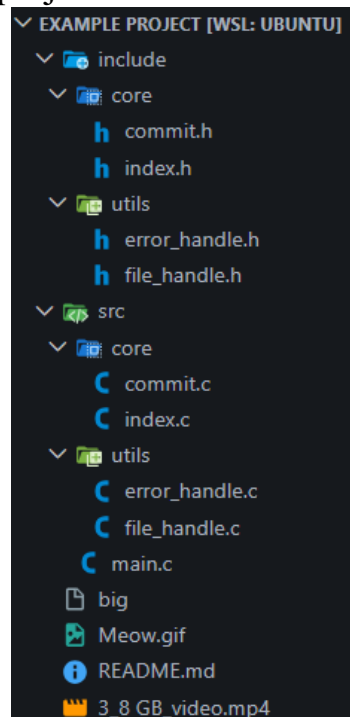
And then move the executable file to your project or add it into environment variables by using this command with your path to executable file in Bash.

```
export PATH="$PATH:/"Your Path"/B.I.G/bin"
```



Here I use the second method for convenience.

This is an example of a project's structure. We will run through with this.



There are all features now in B.I.G.

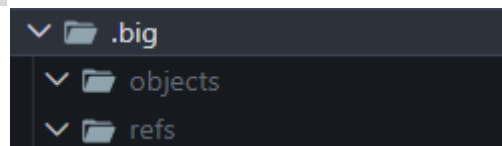
```
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big
Usage: big <command> [-option] [<args>]

Commands:
  big init                Initialize current directory to project root
  big add [-d | --delete] <file or directory> ... Add files into index to wait for commit
  big commit [-m <message>] Commit and enter log
  big log [-<amount>]     Show previous commit logs
  big status              Show the status of files from project root
  big checkout <commit hash or branch name> Change project root to past commit status
  big branch [-d | -delete] [<branch name>] List branches or create branch with name
```

Using **big init** to initialize the project directory.

```
⊙ (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big add
Error: Not an initialized directory. Use 'big init' to initialize current directory
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big init
Start to initialize B.I.G structure...
Directory initialize complete
⊙ (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big init
Error: Directory already initialized. Operation cancelled
```

There will have **.big/** folder and essential folders in current working directory.



Using **big status** to show all files' status in project directory
(Note: Here will slower because the 3.8 GB video)

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
  untracked: 3_8_GB_video.mp4
  untracked: Meow.gif
  untracked: README.md
  untracked: include/core/commit.h
  untracked: include/core/index.h
  untracked: include/utls/error_handle.h
  untracked: include/utls/file_handle.h
  untracked: src/core/commit.c
  untracked: src/core/index.c
  untracked: src/main.c
  untracked: src/utls/error_handle.c
  untracked: src/utls/file_handle.c
```

Using `big add` to stage file into Index (one or multiple file or directory)

```
(base) eender@DESKTOP-Eender:~/Workspace/Example Project/src$ big add core/ main.c utils/
(base) eender@DESKTOP-Eender:~/Workspace/Example Project/src$ big status
Ready to commit:
    new file:   src/core/commit.c
    new file:   src/core/index.c
    new file:   src/main.c
    new file:   src/utils/error_handle.c
    new file:   src/utils/file_handle.c

Changes not staged:
    untracked:  3_8_GB_video.mp4
    untracked:  Meow.gif
    untracked:  README.md
    untracked:  include/core/commit.h
    untracked:  include/core/index.h
    untracked:  include/utils/error_handle.h
    untracked:  include/utils/file_handle.h
```

Using `big commit` to commit staged files and write log in Nano

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project/src$ big commit
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project/src$ big status
Ready to commit:

Changes not staged:
    untracked:  3_8_GB_video.mp4
    untracked:  Meow.gif
    untracked:  README.md
    untracked:  include/core/commit.h
    untracked:  include/core/index.h
    untracked:  include/utils/error_handle.h
    untracked:  include/utils/file_handle.h
```

```
GNU nano 7.2
// Write down your commit log below this line
Feat: My first commit!
```

Or using `big commit -m <"msg">` to fast commit

```
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big add .
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big commit -m "feat: My second log with remain files"
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
```

We can use to see commit history (Default branch is main)

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big log
Branch: main
Commit: 1ad05b44b413d5e5      Date: 2025/12/08 21:37:03      Log: "feat: My second log with remain files"
Commit: c6deba9d079c9f7f     Date: 2025/12/08 21:33:22      Log: "Feat: My first commit! "
```

By using `big branch <branch name>` to create a new branch (but not change to the new branch). `big branch` to show branches

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big branch new_branch
Branch 'new_branch' created. Point to commit: 1ad05b44b413d5e5
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big branch
new_branch
* main
```

Now I modify some files' content and delete a file. Then try to check out to another branch. But there are uncommitted files so operation has been stopped.

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
    modified:   src/core/commit.c
    deleted:    src/core/index.c
    modified:   src/main.c

⦿ (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big checkout new_branch
Error: There are changes not commit. Please commit first or discard changes
```

We commit those files and try again.

Now can safely use `big checkout <branch name>` change to another branch.

You can see the newest log is remain in second commit

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big add .
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big commit -m "fix: Third commit for changed files"
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:

● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big checkout new_branch
Change to branch: new_branch
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:

● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big log
Branch: new_branch
Commit: 1ad05b44b413d5e5      Date: 2025/12/08 21:37:03      Log: "feat: My second log with remain files"
Commit: c6deba9d079c9f7f      Date: 2025/12/08 21:33:22      Log: "Feat: My first commit! "
```

We can also use Commit hash ID as target by using `big checkout <hash>`.

It create a temp branch that will be replaced if change to another hash.

```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big checkout c6deba9d079c9f7f
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:

● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big log
Branch: temp_checkout_ref
Commit: c6deba9d079c9f7f      Date: 2025/12/08 21:33:22      Log: "Feat: My first commit! "
```

Now we back to the main branch first.

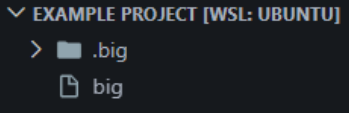
```
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big checkout main
Change to branch: main
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big log
Branch: main
Commit: 66c12a5c4355fb13      Date: 2025/12/08 21:53:02      Log: "fix: Third commit for changed files"
Commit: 1ad05b44b413d5e5      Date: 2025/12/08 21:37:03      Log: "feat: My second log with remain files"
Commit: c6deba9d079c9f7f      Date: 2025/12/08 21:33:22      Log: "Feat: My first commit! "
● (base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
```

Now I delete all data in project directory. We would like to restore all file and see they are still working. Especially the binary file like video.

```
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
  deleted: 3_8_GB_video.mp4
  deleted: Meow.gif
  deleted: README.md
  deleted: include/core/commit.h
  deleted: include/core/index.h
  deleted: include/utls/error_handle.h
  deleted: include/utls/file_handle.h
  deleted: src/core/commit.c
  deleted: src/main.c
  deleted: src/utls/error_handle.c
  deleted: src/utls/file_handle.c
```



Now we add all deleted files then commit.

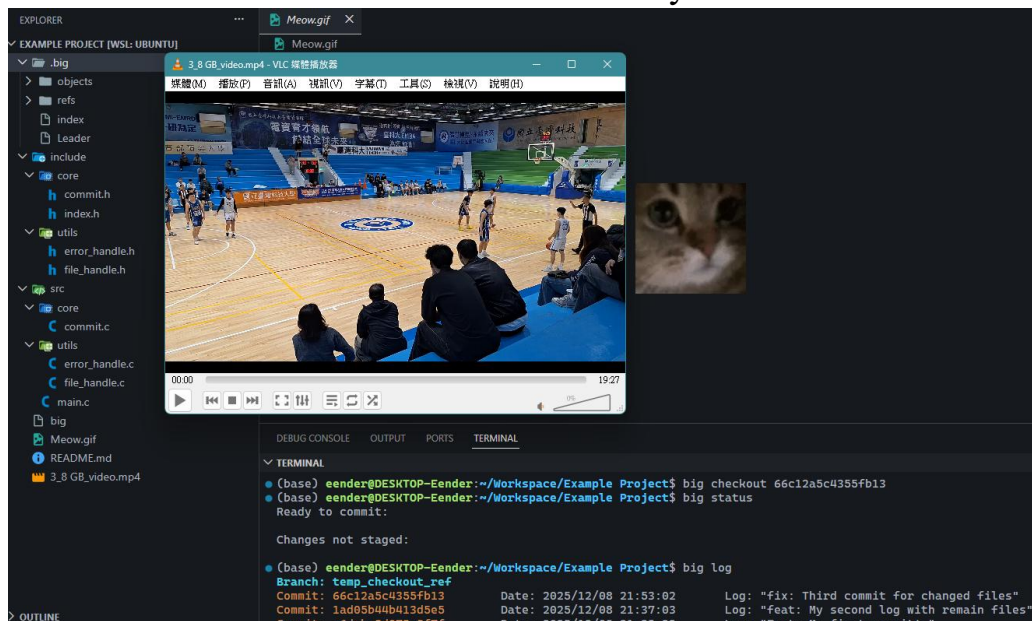
```
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big add .
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:
  deleted: 3_8_GB_video.mp4
  deleted: Meow.gif
  deleted: README.md
  deleted: include/core/commit.h
  deleted: include/core/index.h
  deleted: include/utls/error_handle.h
  deleted: include/utls/file_handle.h
  deleted: src/core/commit.c
  deleted: src/main.c
  deleted: src/utls/error_handle.c
  deleted: src/utls/file_handle.c

Changes not staged:

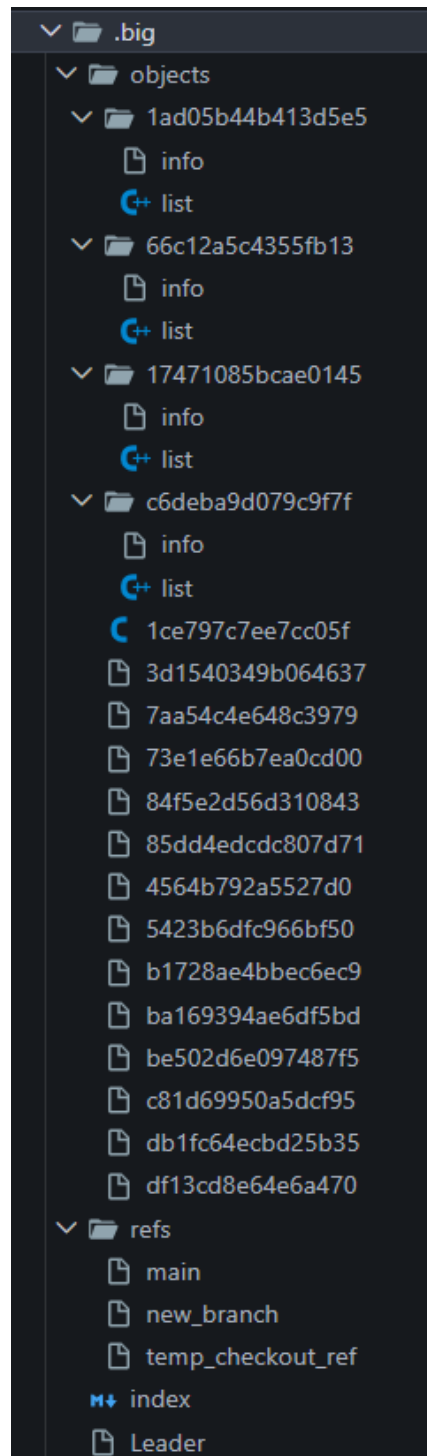
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big commit -m "test: Fourth commit for delete all files"
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big log
Branch: main
Commit: 17471085bcae0145 Date: 2025/12/08 22:45:13 Log: "test: Fourth commit for delete all files"
Commit: 66c12a5c4355fb13 Date: 2025/12/08 21:53:02 Log: "fix: Third commit for changed files"
Commit: 1ad05b44b413d5e5 Date: 2025/12/08 21:37:03 Log: "feat: My second log with remain files"
Commit: c6deba9d079c9f7f Date: 2025/12/08 21:33:22 Log: "Feat: My first commit! "
(base) eender@DESKTOP-Eender:~/Workspace/Example Project$ big status
Ready to commit:

Changes not staged:
```

Checkout to previous commit hash. All files were restored successfully. The video and GIF can normally work.



The final status of `.big/` folder like this.



Here show the Index and Leader store data

```
index
.big > index
1 11
2 3_8_GB_video.mp4 3d1540349b064637 0
3 Meow.gif db1fc64ecbd25b35 0
4 README.md be502d6e097487f5 0
5 include/core/commit.h df13cd8e64e6a470 0
6 include/core/index.h b1728ae4bbec6ec9 0
7 include/utls/error_handle.h 85dd4edcdc807d71 0
8 include/utls/file_handle.h ba169394ae6df5bd 0
9 src/core/commit.c c81d69950a5dcf95 1
10 src/main.c 4564b792a5527d0 1
11 src/utls/error_handle.c 5423b6dfc966bf50 0
12 src/utls/file_handle.c 73e1e66b7ea0cd00 0
13

Leader
.big > Leader
1 temp_checkout_ref
2
```

Branch files

```
temp_checkout_ref
.big > refs > temp_checkout_ref
1 66c12a5c4355fb13
2

new_branch
.big > refs > new_branch
1 1ad05b44b413d5e5
2

main
.big > refs > main
1 17471085bcae0145
2
```

Commit information

```
info
.big > objects > 1ad05b44b413d5e5 > info
1 2025/12/08 21:37:03
2 c6deba9d079c9f7f
3 feat: My second log with remain files
4
```

VIII. Discussion

Performance vs. Accuracy

Currently, `big status` calculates the hash of every file to detect changes. This is 100% accuracy but is slow ($O(n)$ IO operations) for large projects. In future Improvement, we can use timestamp to check the last changed time or check the size of files.

.bigignore

Just like `.gitignore`. This is a feature I think is very useful in a project that uses version control. Maybe will be the first order for future work.

Using `big add` in inner directory

This is the tricky part of `add`. Because it needs to calculate the relative path to the project directory. I was going to give up this because it needs to include many situations like using `big add ..` to add outer directory. I spend lots of time in the early stage of the project to solve this problem.

Many Times of Refactoring

I have refactored the code structure at least three times. From just have source and header files. To split them into different layers. Finally, use Blob and generic data structures to implement the core part. This is my most accomplished part of project.

IX. Conclusion

B.I.G is really the biggest project I have ever done. Through this project, have gained a deeper mastery of the C language and system development.

1. Memory Management

My initial approach of loading entire files into memory wasted lots of memory space and copy files as commit wasted disk space. By implementing Content-Addressable Storage (CAS) and handling memory lifecycles manually (`malloc` / `free`), I learned how to build memory-efficient systems that can handle large files without leaking resources.

2. Data Structure Application

I turn textbook concepts into a real project. More importantly, I learned how to split the data structure and Business Logic. And implement generic data structure for future use. The point that I did not use real graph structure was a little pity.

3. Structure of code and system

I learned the importance of separation of concerns. Initially, my code needed to handle everything in a .c file. By splitting the work into different files. Make the project more readable and clear division of work.

4. Headers in standard library

The functions in standard library really have many points need to know. How do they work? Error handling and return value are important. I use lots of I/O in this project and gave me a understanding of how the OS handles files.

B.I.G is now a fully functional, binary-safe version control system that I built from scratch.